

Assignment No. 5

Title

Implement Greedy search algorithm.

Aim

Implement Greedy search algorithm for Selection Sort.

Objectives

- To understand concept of Greedy Search.
- To understand concept of Selection Sort.
- To Implement Selection Sort.
- To analyze the time complexity of selection sort algorithm.

Theory

What Is Greedy Algorithm?

Greedy algorithm is a problem-solving strategy that makes locally optimal decisions at each stage in the hopes of achieving a globally optimum solution. This simple, intuitive algorithm can be applied to solve any optimization problem which requires the maximum or minimum optimum result. The best thing about this algorithm is that it is easy to understand and implement.

The runtime complexity associated with a greedy solution is pretty reasonable. However, you can implement a greedy solution only if the problem statement follows two properties mentioned below:

- Greedy Choice Property: Choosing the best option at each phase can lead to a global (overall) optimal solution.
- Optimal Substructure: If an optimal solution to the complete problem contains the optimal solutions to the sub problems, the problem has an optimal substructure.

Moving forward, we will learn how to create a greedy solution for a problem that adheres to the principles listed above.

Steps for Creating a Greedy Algorithm

By following the steps given below, you will be able to formulate a greedy solution for the given problem statement:

- Step 1: In a given problem, find the best substructure or sub problem.
- Step 2: Determine what the solution will include (e.g., largest sum, shortest path).
- Step 3: Create an iterative process for going over all sub problems and creating an optimum solution

In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. It is also the simplest algorithm. It is an in-place comparison sorting algorithm. In this algorithm, the array is divided into two parts, first is sorted part, and another one is the unsorted part. Initially, the sorted part of the array is empty, and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.

How Selection Sort Works?

Consider the following depicted array as an example.



For the first position in the sorted list, the whole list is scanned sequentially. The first position where 14 is stored presently, we search the whole list and find that 10 is the lowest value.



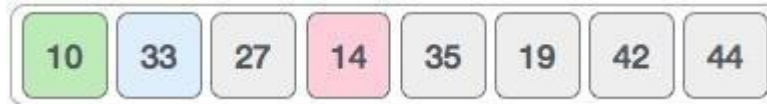
So we replace 14 with 10. After one iteration 10, which happens to be the minimum value in the list, appears in the first position of the sorted list.



For the second position, where 33 is residing, we start scanning the rest of the list in a linear manner.



We find that 14 is the second lowest value in the list and it should appear at the second place. We swap these values.

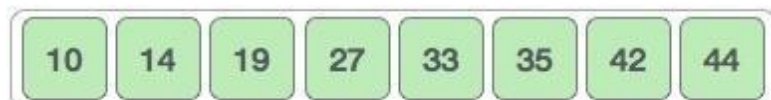
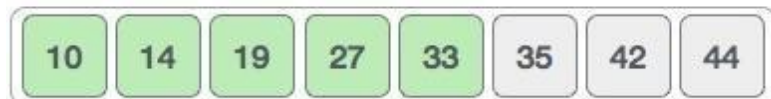
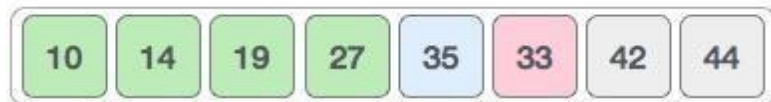
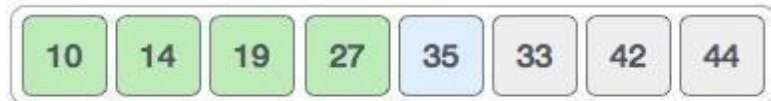
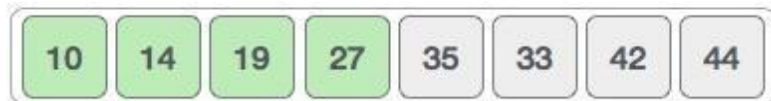


After two iterations, two least values are positioned at the beginning in a sorted manner.



The same process is applied to the rest of the items in the array.

Following is a pictorial depiction of the entire sorting process –



Algorithm

Step 1 – Set MIN to location 0

Step 2 – Search the minimum element in the list

Step 3 – Swap with value at location MIN

Step 4 – Increment MIN to point to next element

Step 5 – Repeat until list is sorted

Pseudocode

Procedure selection sort

list: array of items

n : size of list

for i = 1 to n - 1

/* set current element as minimum*/

min = i

/* check the element to be minimum */

for j = i+1 to n

if list[j] < list[min] then

min = j;

end if

end for

/* swap the minimum element with the current element*/

if indexMin != i then

swap list[min] and list[i]

end if

end for

end procedure

Time Complexity

Best Case

$O(n^2)$

Average Case

$O(n^2)$

Worst Case

$O(n^2)$

Space Complexity

$O(1)$

Conclusion

This program gives us the knowledge tree data structure.

Questions:

1. What is Greedy Search Strategy?
2. Explain selection sort with example?
3. What is Time complexity and space complexity of selection sort.
4. Write an algorithm for selection sort.
5. write Pseudocode for selection sort